Planificación automática aplicada al Senku

Gregorio Morillo Zamora
Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
XVJ8387 - gremorzam@alum.us.es

Raúl Jimenez Pérez
Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
LSW694 - raujimper@alum.us.es

***Resumen*—En este trabajo se aborda la resolución del juego de solitario Senku mediante técnicas de planificación automática. Se modela el dominio empleando el lenguaje PDDL con una única acción parametrizada sobre tripletas de casillas adyacentes, y se implementan dos algoritmos de búsqueda en espacio de estados sobre la representación generada por la librería Unified Planning: una búsqueda básica que explora sistemáticamente el árbol de juego y una búsqueda en haz (beam search) guiada por la heurística de función pagoda. La experimentación sobre cinco configuraciones de tablero (33–41 casillas) demuestra que la búsqueda básica resuelve el tablero clásico tras expandir 1,93 millones de nodos, mientras que la búsqueda en haz con pagoda dual permite resolver los tableros clásico, variante 1 y variante 3 en tiempos comprendidos entre 44 s y 204 s. La variante 5, con 41 casillas, no pudo resolverse con ninguna combinación de pagoda, lo que sugiere que la heurística basada exclusivamente en paridad de coordenadas resulta insuficiente para guiar la búsqueda en espacios de estado de mayor tamaño.**

***Palabras clave*—Inteligencia Artificial, Planificación Automática, Senku, PDDL, Búsqueda en Haz, Función Pagoda, Unified Planning.**

I. Introducción

El juego del Senku, también conocido como solitario o Peg Solitaire, es un pasatiempo de tablero para un solo jugador que ha demostrado ser un entorno particularmente fértil para la experimentación en Inteligencia Artificial. Su carácter determinista, la existencia de un espacio de estados bien definido y la posibilidad de aplicar heurísticas basadas en la geometría del tablero lo convierten en un problema de referencia para evaluar el comportamiento de agentes de planificación automática y algoritmos de búsqueda en espacio de estados [1]. El objetivo del juego consiste en reducir el número de piezas del tablero hasta dejar una única pieza, idealmente en la posición del hueco original, mediante saltos ortogonales sobre piezas adyacentes que son eliminadas durante el proceso.

El objetivo principal de este trabajo es diseñar un sistema capaz de representar formalmente el problema del Senku y resolverlo de forma automática aplicando algoritmos de búsqueda. Para alcanzar este objetivo, la propuesta didáctica de la asignatura [4] establece varios hitos específicos que estructuran el desarrollo del proyecto:

- Expresar formalmente el problema del Senku en un lenguaje de descripción de dominios, en nuestro caso PDDL, capturando la geometría del tablero y las reglas de movimiento mediante predicados y acciones.
- Implementar en Python un motor de búsqueda básico (BFS/DFS) capaz de explorar el árbol de juego y encontrar soluciones para tableros de tamaño reducido y para el tablero clásico de 33 casillas.
- Diseñar e implementar un algoritmo de búsqueda informada, en concreto búsqueda en haz (beam search), guiado por la heurística de función pagoda, que permita resolver configuraciones de tablero más exigentes: las variantes 1, 3 y 5 definidas en la propuesta del proyecto.
- Documentar el proceso completo de diseño, implementación y experimentación siguiendo un formato de artículo científico, y realizar una presentación oral del trabajo realizado.

El enfoque adoptado para este trabajo ha combinado el estudio teórico de las técnicas de planificación vistas en la asignatura con un desarrollo iterativo guiado por la experimentación. Como herramienta de apoyo se ha utilizado inteligencia artificial generativa para la generación de bocetos iniciales de código, la depuración de errores y la redacción de secciones de la memoria, siguiendo las directrices de uso responsable establecidas en la propuesta [4]. Todas las contribuciones generadas por estos sistemas han sido revisadas, adaptadas y verificadas por los autores, y las conversaciones completas están documentadas en los enlaces incluidos en el apartado de referencias [6]–[12].

La memoria se organiza de la siguiente manera. La sección II presenta los fundamentos teóricos necesarios para comprender el desarrollo: el juego del Senku, el lenguaje PDDL, los conceptos de búsqueda en espacio de estados, la función pagoda como heurística y el algoritmo de búsqueda en haz. La sección III describe en detalle la metodología empleada, incluyendo el modelado del dominio en PDDL, la implementación de los dos algoritmos de búsqueda y las decisiones de diseño adoptadas para superar las dificultades encontradas. La sección IV recoge los resultados experimentales obtenidos sobre las cinco configuraciones de tablero, junto con un análisis comparativo. Finalmente, la sección V extrae las conclusiones del trabajo.

II. Preliminares

II-A. El juego del Senku

El Senku es un juego de tablero para un solo jugador que comienza con un número M de piezas dispuestas sobre una

cuadrícula y un único hueco libre. En cada turno, el jugador selecciona una pieza y la desplaza en dirección horizontal o vertical haciéndola saltar sobre otra pieza adyacente para aterrizar en una casilla vacía inmediatamente posterior. La pieza sobre la que se ha saltado es retirada del tablero. Los movimientos en diagonal no están permitidos.

El objetivo del juego es reducir el tablero hasta que quede una única pieza. En la formulación clásica, esta pieza final debe ocupar la misma posición en la que se encontraba el hueco inicial, aunque en este trabajo se adopta una definición más flexible que considera como solución cualquier configuración con una sola pieza, independientemente de su posición [4].

Cada partida de Senku se puede representar como un árbol de juego en el que los nodos son las posibles configuraciones del tablero y las aristas son movimientos válidos que transforman un estado en otro. Esta estructura hace que los algoritmos de búsqueda en grafos sean una herramienta natural para explorar el espacio de soluciones.

II-B. Planificación automática y PDDL

La planificación automática es una rama de la Inteligencia Artificial que estudia cómo generar secuencias de acciones que permitan alcanzar un estado objetivo a partir de un estado inicial, dada una descripción formal del entorno y de las acciones disponibles [3].

PDDL (Planning Domain Definition Language) es el lenguaje estándar para describir problemas de planificación. En PDDL se separa la descripción del dominio (conceptos generales, predicados y acciones) de la descripción del problema (objetos concretos, estado inicial y condición objetivo). Esta separación permite reutilizar el mismo dominio para múltiples instancias del problema. Para este trabajo se utiliza el subconjunto :strips del lenguaje, que permite expresar acciones mediante precondiciones y efectos lógicos.

II-C. Búsqueda en espacio de estados

Dado un problema de planificación, se puede construir un grafo dirigido donde cada nodo representa un estado del mundo y cada arista la aplicación de una acción que transforma un estado en otro. Los algoritmos de búsqueda en espacio de estados exploran este grafo partiendo del estado inicial hasta encontrar un nodo que satisfaga la condición objetivo.

Dentro de la búsqueda no informada, las dos estrategias fundamentales son:

- **Búsqueda en anchura (BFS):** explora el árbol por niveles, expandiendo primero todos los nodos a una misma profundidad antes de pasar al nivel siguiente. Garantiza la solución de menor número de pasos, pero su consumo de memoria crece exponencialmente.
- **Búsqueda en profundidad (DFS):** sigue un único camino hasta agotarlo antes de retroceder, utilizando menos memoria que BFS. Sin embargo, no garantiza optimalidad y puede perderse en ramas muy profundas.

Ambas estrategias requieren un mecanismo para detectar estados repetidos — normalmente un conjunto de visitados— que evite la generación de ciclos.

II-D. Función pagoda

La función pagoda es una heurística específica para el Senku que asigna un peso numérico $p(c)$ a cada casilla c del tablero [1]. La suma de los pesos de todas las casillas ocupadas en un estado constituye el valor pagoda de dicho estado. Para que la función sea admisible como heurística debe cumplir la condición de que, para cualquier tripleta de casillas adyacentes (a, b, c) —donde a es el origen, b la intermedia y c el destino de un salto válido— se verifique que $p(a) + p(b) \geq p(c)$. Esta propiedad asegura que el valor de pagoda nunca aumenta tras realizar un movimiento, por lo que es monótona decreciente y puede emplearse como cota inferior del esfuerzo restante.

En este trabajo se han implementado tres variantes de asignación de pesos, basadas en la paridad de las coordenadas (x, y) de cada casilla en la convención de nombrado cXY:

- **V1:** asigna peso 1 a las casillas con x impar e y impar.
- **V2:** asigna peso 1 a las casillas con x par e y par.
- **V3:** asigna peso 1 a las casillas con x par e y impar.

El resto de casillas reciben peso 0. Cada variante induce una heurística distinta que prioriza la eliminación de piezas en determinadas zonas del tablero.

II-E. Algoritmo Beam Search

La búsqueda en haz (beam search) es una variante de la búsqueda en anchura que incorpora una heurística para limitar la expansión del árbol en cada nivel. En lugar de conservar todos los nodos de un nivel, el algoritmo mantiene únicamente los W nodos más prometedores según la función heurística, donde W es un parámetro conocido como *anchura del haz* o *beam width* [2].

El procedimiento general es el siguiente: partiendo del nodo inicial, se generan todos sus sucesores y se evalúan con la heurística. Los W sucesores con mejor valor pasan a formar la frontera del siguiente nivel. En cada iteración se repite el proceso: se expanden todos los nodos de la frontera actual, se evalúan todos los sucesores generados y se conservan los W mejores. Si en algún momento se alcanza un estado que cumple la condición de objetivo, el algoritmo termina y devuelve la secuencia de acciones.

Al podar el árbol en cada nivel, beam search consigue mantener un uso de memoria acotado y controlable, pero a costa de poder perder la solución si esta pasa por estados que no se encuentran entre los W mejores de su nivel. La elección del valor de W y de la función heurística es, por tanto, determinante para el éxito del algoritmo.

II-F. Trabajo relacionado

El Senku ha sido objeto de estudio en la literatura de Inteligencia Artificial desde hace décadas. Beasley [1] realizó un análisis exhaustivo del juego empleando técnicas de búsqueda con función pagoda, demostrando que todas las posiciones del tablero inglés estándar pueden resolverse con la heurística adecuada. La librería Unified Planning [5] proporciona una implementación de referencia para la manipulación de problemas PDDL en Python, incluyendo un simulador fiel para la ejecución de acciones sobre los estados. El presente trabajo

se apoya en ambas contribuciones para construir un sistema que integre el modelado PDDL con algoritmos de búsqueda personalizados.

III. Metodología

III-A. Modelado del dominio en PDDL

III-A1. Predicados y acción saltar: El dominio PDDL define tres predicados y una única acción. Los predicados son: `(pieza ?c)`, que indica que una casilla está ocupada; `(vacio ?c)`, que indica que está libre; y `(adyacente ?c1 ?c2 ?c3)`, que codifica una tripleta de casillas alineadas en línea recta (horizontal o vertical) donde desde `?c1` se puede saltar sobre `?c2` para aterrizar en `?c3`.

La acción `saltar` recibe tres parámetros: la casilla origen (donde se encuentra la pieza que salta), la casilla intermedia (sobre la que se salta) y la casilla destino (donde se aterriza). Las precondiciones exigen que el origen y la intermedia estén ocupados por piezas, que el destino esté vacío y que las tres casillas formen una tripleta adyacente válida. Los efectos actualizan los tres predicados: origen e intermedia pasan a estar vacíos y destino pasa a estar ocupado.

Una decisión de diseño notable es que la relación `adyacente` se modela como un predicado estático que se define explícitamente en el archivo de cada problema. Esto evita tener que calcular geometrías dentro del dominio PDDL, pero genera una gran cantidad de fluentes estáticos en los problemas —76 adyacencias en el tablero clásico— que deben ser filtrados durante la búsqueda para no degradar el rendimiento. El dominio completo requiere únicamente el subconjunto `:strips` de PDDL.

III-A2. Variantes de tablero: Se han definido cinco problemas PDDL que comparten el mismo dominio:

- **Prueba (3 piezas):** tablero reducido de 33 casillas con únicamente tres piezas y treinta huecos. Diseñado para verificar la corrección de los algoritmos en un caso simple.
- **Tablero clásico:** 33 casillas (32 piezas, 1 hueco central) con la disposición en cruz del Senku inglés tradicional.
- **Variante 1:** 37 casillas (36 piezas, 1 hueco), que añade cuatro casillas respecto a la clásico en las esquinas cóncavas interiores.
- **Variante 3:** 39 casillas (38 piezas, 1 hueco), que extiende aún más el tablero incorporando filas adicionales.
- **Variante 5:** 41 casillas (40 piezas, 1 hueco), la configuración más grande, con una topología irregular.

En todos los tableros el hueco inicial se encuentra en la casilla central (generalmente `c44`) y el objetivo, en su versión estricta del PDDL, exige que la pieza final ocupe esa misma posición. No obstante, en la experimentación se ha adoptado un criterio más flexible (`goal_anywhere`) que considera válido cualquier estado con una única pieza, independientemente de su ubicación.

III-B. Búsqueda básica en espacio de estados

III-B1. Implementación y optimizaciones: El primer algoritmo implementado consiste en una exploración sistemática del árbol de juego que admite dos modos de operación (BFS y DFS). La implementación mantiene dos estructuras fundamentales: una frontera que almacena los nodos pendientes de explorar y un conjunto de visitados que evita ciclos. Cada nodo almacena el propio estado, una referencia al nodo padre, la acción que generó el estado y la profundidad en el árbol. Cuando se alcanza un nodo objetivo, la secuencia de acciones se reconstruye recorriendo las referencias al padre hasta la raíz.

Durante el desarrollo se identificaron dos problemas principales de rendimiento que requirieron soluciones de diseño específicas:

Generación de movimientos. La librería Unified Planning proporciona un simulador que permite obtener las acciones aplicables en un estado mediante `get_applicable_actions`. Sin embargo, esta operación realiza una instanciación completa de todas las combinaciones posibles de parámetros, lo que para el tablero clásico supone evaluar más de treinta mil combinaciones en cada nodo, con un coste de decenas de segundos por llamada. Para resolver este problema se optó por extraer las relaciones de adyacencia una única vez al inicio de la búsqueda y, a partir de entonces, generar los movimientos válidos recorriendo directamente la lista de adyacencias. Para cada tripleta $(\text{origen}, \text{intermedia}, \text{destino})$ se comprueba si la casilla origen contiene una pieza, la intermedia contiene una pieza y el destino está vacío. Esta comprobación se reduce a tres accesos booleanos en memoria, reduciendo el coste de generación de movimientos de decenas de segundos a microsegundos.

Cálculo del identificador hash. El conjunto de visitados utiliza un `frozenset` con los fluentes activos como identificador de cada estado. Inicialmente se construía este conjunto recorriendo todos los fluentes definidos en el problema, incluyendo los del predicado `adyacente` que son estáticos y nunca cambian. Para el tablero clásico, esto suponía procesar más de 150 fluentes por estado, la mayoría de ellos invariantes. La optimización consistió en filtrar los fluentes al inicio, conservando únicamente los correspondientes a los predicados `pieza` y `vacio` —que son los que realmente cambian durante la partida—, lo que reduce los fluentes a evaluar a un máximo de 33 (uno por casilla) y acelera significativamente cada comprobación.

Estas optimizaciones permitieron alcanzar una velocidad del orden de cientos de nodos por segundo y hacer viable la resolución completa del tablero clásico.

III-B2. Pseudocódigo: La figura 1 presenta el pseudocódigo del algoritmo de búsqueda básica. La función `elegirNodo` determina la estrategia: en BFS extrae el primer elemento de la frontera (comportamiento FIFO), mientras que en DFS extrae el último (comportamiento LIFO). La función `generarMovimientos` recorre las adyacencias precomputadas comprobando las condiciones de pieza y hueco en cada tripleta. La función `hash` construye el identificador inmutable del estado a partir de los fluentes activos.

```
busquedaBasica(estadoInicial, usarBFS)
Entrada: estado inicial, modo de búsqueda
Salida: plan de acciones o None
frontera ← [NODO(estadoInicial)]
visitados ← { HASH(estadoInicial) }
mientras frontera ≠ ∅ hacer
    nodo ← ELEGIRNODO(frontera, usarBFS)
    acciones ← GENERARMOVIMIENTOS(nodo.estado)
    para cada acc en acciones hacer
        sucesor ← APLICAR(acc, nodo.estado)
        si HASH(sucesor) ∉ visitados entonces
            si ESOBJETIVO(sucesor) entonces
                devolver RECONSTRUIRPLAN(sucesor)
            visitados ← visitados ∪ { HASH(sucesor) }
            frontera ← frontera ∪ [NODO(sucesor, nodo, acc)]
        fin si
    fin para
fin mientras
devolver None
```

Figura 1. Algoritmo de búsqueda básica (BFS/DFS) para el Senku

III-C. Búsqueda en haz (Beam Search)

III-C1. Adaptación al Senku: La implementación de la búsqueda en haz reutiliza directamente las optimizaciones desarrolladas para la búsqueda básica —generación de movimientos mediante adyacencias precomputadas, filtrado de fluentes relevantes y cálculo eficiente de hash—, por lo que no se reprodujeron los problemas de rendimiento que sí aparecieron en aquella. La estructura del algoritmo es similar, pero en lugar de conservar todos los sucesores de un nivel, los ordena según la heurística pagoda y retiene únicamente los W mejores.

La función principal recibe como parámetros la instancia del problema, la anchura del haz W, la variante de pagoda a emplear, y un indicador `goal_anywhere` que, cuando está activo, considera como objetivo cualquier estado con una única pieza (en lugar de la posición concreta definida en el PDDL).

Un problema específico que surgió durante el desarrollo fue la falta de discriminación de la heurística pagoda simple cuando muchos estados en un mismo nivel comparten el mismo valor. La pagoda V1 tiene únicamente 12 valores posibles distintos (tantos como casillas con peso 1 en el tablero clásico), por lo que dentro de cada nivel los nodos se ordenan esencialmente de forma arbitraria entre aquellos con el mismo valor pagoda. Esto provocaba que, en tableros grandes, el haz perdiera el camino correcto en niveles profundos.

La solución adoptada fue introducir criterios de desempate mediante pagoda dual y triple. Cuando se proporciona una pagoda secundaria, la ordenación de los sucesores utiliza una tupla (H_1, H_2) donde H_1 es el valor de la pagoda primaria y H_2 el de la secundaria. Esto aumenta el número de bins de ordenación de aproximadamente 12 a más de 144 combinaciones. Para los casos más exigentes se implementó la ordenación triple (H_1, H_2, H_3), elevando el número de bins a más de 1.700. Esta extensión permitió resolver tableros que con la pagoda simple eran intratables.

III-C2. Pseudocódigo: El pseudocódigo del algoritmo de búsqueda en haz se muestra en la figura 2. La función `ordenarPorPagoda` admite tanto una variante simple como una lista de variantes. La poda del haz se realiza mediante la selección de los primeros W elementos tras la ordenación.

IV. Experimentación y resultados

IV-A. Configuración experimental

Todos los experimentos se han ejecutado sobre un mismo equipo con procesador AMD Ryzen 7 y 32 GB de RAM, utilizando Python 3.12 y la librería Unified Planning 4. Las métricas registradas para cada ejecución han sido: número de nodos expandidos, profundidad de la solución (en número de saltos), tiempo de ejecución y resultado (éxito o fracaso). Para la búsqueda en haz se ha empleado la modalidad `goal_anywhere`, que acepta como objetivo cualquier estado con una única pieza.

IV-B. Verificación con búsqueda básica

La búsqueda básica se verificó en primer lugar sobre el tablero de prueba de 3 piezas, confirmando que tanto BFS como DFS encuentran la solución óptima con 2 movimientos y expandiendo únicamente 3 nodos (BFS) o 2 nodos (DFS).

Posteriormente se ejecutó DFS sobre el tablero clásico completo sin límite de profundidad. Dado que BFS resulta inviable en este tablero por su consumo explosivo de memoria, se optó por DFS, que requiere mantener solo una rama del árbol en la frontera. La búsqueda exploró 1.929.901 nodos durante aproximadamente 90 minutos y encontró una solución de 31 saltos. Este resultado, aunque positivo, evidencia la necesidad de una heurística que guíe la búsqueda hacia estados más prometedores para hacer el proceso práctico.

Los valores de la función pagoda para el estado inicial de cada tablero se recogen en la tabla I. Sirvieron como referencia para calibrar la heurística: un valor pagoda alto indica una configuración de piezas más desfavorable desde el punto de vista de esa variante concreta.

```
busquedaHaz(estadoInicial, W, variantesPagoda)
Entrada: estado inicial, anchura del haz, lista de variantes pagoda
Salida: plan de acciones o None
frontera ← [NODO(estadoInicial)]
visitados ← { HASH(estadoInicial) }
mientras frontera ≠ ∅ hacer
    sucesores ← []
    para cada nodo en frontera hacer
        acciones ← GENERARMOVIMIENTOS(nodo.estado)
        para cada acc en acciones hacer
            sucesor ← APLICAR(acc, nodo.estado)
            si HASH(sucesor) ∉ visitados entonces
                si ESOBJETIVO(sucesor) entonces
                    devolver RECONSTRUIRPLAN(sucesor)
                visitados ← visitados ∪ { HASH(sucesor) }
                sucesores ← sucesores ∪ [NODO(sucesor, nodo, acc)]
            fin si
        fin para
    fin para
    ORDENARPORPAGODA(sucesores, variantesPagoda)
    frontera ← sucesores[:W]
fin mientras
devolver None
```

Figura 2. Algoritmo de búsqueda en haz (Beam Search)

Cuadro I
VALORES PAGODA DEL ESTADO INICIAL POR TABLERO Y VARIANTE

Tablero	V1	V2	V3
Clásico (33)	12	4	8
Variante 1 (37)	12	9	7
Variante 3 (39)	12	6	10
Variante 5 (41)	12	12	8

IV-C. Resultados de Beam Search

La experimentación con beam search se realizó buscando la combinación óptima de variante(s) de pagoda y anchura de haz para cada tablero. La estrategia consistió en probar las variantes V1, V2 y V3 como pagoda primaria, y combinarlas con pagodas secundarias (y terciaria) cuando la simple resultaba insuficiente.

IV-C1. Tablero clásico (33 casillas): Con la pagoda V1 como único criterio de ordenación y una anchura de haz $W = 500$, el algoritmo encuentra una solución de 31 saltos tras expandir 11.667 nodos en 44 s. Durante los primeros niveles el haz se satura rápidamente (500 nodos por nivel desde el nivel 5), para comenzar a reducirse a partir del nivel 26 hasta alcanzar el objetivo en el nivel 31.

La tabla II resume los resultados de todos los tableros. La solución encontrada para el clásico requiere 31 movimientos, comenzando con un salto que desplaza la pieza de la fila superior hacia el hueco central, y concluye con la eliminación de las últimas piezas en la zona central del tablero.

IV-C2. Variante 1 (37 casillas): La pagoda V1 como único criterio resultó insuficiente para guiar la búsqueda en este tablero de mayor tamaño. Fue necesario emplear una pagoda secundaria V3 como criterio de desempate, formando la combinación V1+V3, y aumentar la anchura del haz a $W = 1000$. Con esta configuración se encontró una solución de 35 saltos tras expandir 26.763 nodos en 138 s.

La elección de V3 como secundaria (y no la V2, que es la que se asigna por defecto de forma automática) se determinó experimentalmente: la combinación V1+V2 no conseguía mantener el haz lo suficientemente diversificado en los niveles intermedios, mientras que V1+V3 proporcionaba una mejor discriminación.

IV-C3. Variante 3 (39 casillas): La variante 3 supuso un desafío mayor. Con pagoda simple V3 y $W = 1500$, la búsqueda alcanzaba una profundidad de 34 niveles y se colapsaba a un único nodo sin poder progresar, debido a la falta de diversidad en la frontera. La solución fue emplear V3 como pagoda primaria y V1 como secundaria (combinación V3+V1), con $W = 1500$. Esto permitió que el haz mantuviera suficientes nodos en los niveles críticos (entre $d = 30$ y $d = 36$ se pasa de 1500 nodos a 3) y se encontró una solución de 37 saltos tras expandir 42.194 nodos en 204 s.

IV-C4. Variante 5 (41 casillas): La variante 5, con 41 casillas, resultó ser la más compleja. Se probaron todas las combinaciones de pagoda simple (V1, V2, V3 por separado), dual (V1+V2, V1+V3, V2+V3) y triple (V1+V2+V3), con anchuras de haz entre 500 y 3000. En todos los casos la búsqueda alcanza la profundidad 36 (de los 39 movimientos que requiere teóricamente una solución), donde sobreviven entre 33 y 42 nodos con exactamente 4 piezas cada uno y ningún movimiento válido disponible.

La configuración más exigente probada fue V1+V2+V3 con $W = 3000$, que expandió 86.928 nodos en 598 s y alcanzó la profundidad 36, pero no pudo progresar más allá. Este comportamiento sugiere una limitación fundamental de la heurística basada en paridad: las asignaciones de peso 0/1 no proporcionan suficiente información para discriminar estados

Cuadro II
RESULTADOS COMPLETOS DE BEAM SEARCH

Tablero	Nodos	Pasos	Tiempo	Pagoda(s)
Clásico (33)	11.667	31	44 s	V1
Variante 1 (37)	26.763	35	138 s	V1+V3
Variante 3 (39)	42.194	37	204 s	V3+V1
Variante 5 (41)	86.928	–	598 s	V1+V2+V3

Cuadro III
COMPARATIVA ENTRE BÚSQUEDA BÁSICA Y BEAM SEARCH SOBRE EL TABLERO CLÁSICO

	DFS	Beam Search (V1)
Nodos expandidos	1.929.901	11.667
Profundidad (pasos)	31	31
Tiempo	5.429 s	44 s
Anchura de haz	—	500

en un tablero de 41 casillas con la geometría irregular de esta variante.

IV-D. Análisis global

La tabla II presenta los resultados globales. Se observa una correlación directa entre el número de casillas del tablero y la dificultad de resolución: a mayor número de casillas, mayor es el número de nodos expandidos y el tiempo requerido. La excepción es la variante 5, que no pudo resolverse a pesar de haber empleado la configuración más potente (triple pagoda y $W = 3000$).

En términos de eficacia, el algoritmo resuelve 3 de los 4 tableros propuestos para la evaluación (clásico, variante 1 y variante 3), con tiempos que oscilan entre 44 segundos y algo más de 3 minutos. La profundidad de las soluciones encontradas aumenta con el tamaño del tablero: 31 pasos para el clásico (33 casillas), 35 para la variante 1 (37) y 37 para la variante 3 (39).

La principal limitación observada es que el haz se satura en los niveles tempranos (entre 5 y 10, dependiendo del tablero), alcanzando el máximo de W nodos en todos los niveles intermedios. Esto significa que la poda es muy agresiva y que la diversidad de la frontera depende críticamente de la capacidad de la heurística para diferenciar estados dentro del mismo nivel. La incorporación de pagoda dual y triple como criterio de ordenación mejoró significativamente esta diversidad, pero no fue suficiente para el tablero más grande.

En términos de coste computacional, la tabla III compara directamente ambas estrategias sobre el tablero clásico. La búsqueda básica (DFS) expandió 1.929.901 nodos en aproximadamente 90 minutos, mientras que beam search con pagoda V1 resolvió el mismo tablero expandiendo tan solo 11.667 nodos en 44 s —una reducción de dos órdenes de magnitud tanto en nodos expandidos como en tiempo—, lo que demuestra la efectividad de la heurística pagoda para guiar la búsqueda. Ambos algoritmos encontraron soluciones de la misma profundidad (31 saltos), si bien no necesariamente la misma secuencia de movimientos.

V. CONCLUSIONES

En este trabajo se ha desarrollado un sistema completo de planificación automática para el juego del Senku, cubriendo todas las fases del proceso: modelado del dominio en PDDL, implementación de algoritmos de búsqueda no informada (BFS/DFS) e informada (beam search con heurística pagoda), y experimentación sobre cinco configuraciones de tablero con diferente número de casillas.

La principal conclusión del trabajo es que la combinación de una representación PDDL optimizada con un algoritmo de búsqueda en haz guiado por función pagoda permite resolver el Senku clásico y sus variantes de hasta 39 casillas en tiempos prácticos. La incorporación del criterio de desempate mediante pagoda dual (y triple) como clave de ordenación de los sucesores en cada nivel ha sido la innovación más relevante, ya que ha permitido resolver la variante 3 y la variante 1, tableros que con pagoda simple resultaban intratables.

Los resultados obtenidos ponen de manifiesto la importancia crítica de las heurísticas en problemas de búsqueda con espacios de estado de gran tamaño. Mientras que la búsqueda ciega DFS requiere casi dos millones de expansiones y más de hora y media para resolver el tablero clásico, la incorporación de una heurística tan sencilla como la función pagoda reduce el esfuerzo a poco más de diez mil nodos y menos de un minuto. Esta diferencia evidencia que es más importante una buena optimización que unos buenos recursos: sin información que guíe la exploración, incluso problemas relativamente pequeños como el Senku pueden resultar intratables en la práctica.

El análisis cualitativo revela que la función pagoda basada exclusivamente en la paridad de coordenadas, si bien es computacionalmente muy ligera, presenta un poder de discriminación limitado que se manifiesta especialmente en tableros de geometría irregular y mayor número de casillas. La variante 5 no pudo resolverse con ninguna combinación probada, lo que sugiere que la heurística carece de la granularidad necesaria para distinguir estados prometedores de no prometedores en tableros de 41 casillas. Una función pagoda con más variantes de peso, o una heurística distinta que incorporara información topológica, podría superar esta limitación.

Como trabajo futuro cabría explorar el uso de pagodas con asignaciones de peso más refinadas —incluyendo pesos mayores que 1 y valores no enteros—, el diseño de una batería de pruebas unitarias automatizadas que registren métricas de rendimiento precisas (tiempo por nodo, tasa de expansión, memoria utilizada) para facilitar la comparación sistemática entre configuraciones, y el desarrollo de una interfaz gráfica que permita visualizar la evolución de la búsqueda y las soluciones encontradas.

REFERENCIAS

[1] J. D. Beasley, *The Ins and Outs of Peg Solitaire*. Oxford University Press, 1985.

[2] S. Russell y P. Norvig, *Inteligencia Artificial: Un Enfoque Moderno*, 3ª ed. Pearson, 2016.

[3] M. Ghallab, D. Nau y P. Traverso, *Automated Planning: Theory and Practice*. Morgan Kaufmann, 2004.

[4] G. Chaves Benítez, “Planificación automática aplicada al Senku — Propuesta de trabajo”, Dpto. Ciencias de la Computación e Inteligencia Artificial, Universidad de Sevilla, curso 2025–2026.
[5] Unified Planning Team, “Unified Planning: A Python Library for Planning Domain Definition Language”. [En línea]. Disponible: https://unified-planning.readthedocs.io/
[6] Google Gemini, “Plan de Tareas — Senku-IA”, 2025. [En línea]. Disponible: https://gemini.google.com/share/d61bfccb807f
[7] Google Gemini, “Reglas del Senku — Modelado PDDL”, 2025. [En línea]. Disponible: https://gemini.google.com/share/7347390cccd4
[8] Google Gemini, “PDDL del dominio Senku”, 2025. [En línea]. Disponible: https://gemini.google.com/share/52a398704ec8
[9] Anthropic Claude, “PDDL del dominio Senku”, 2025. [En línea]. Disponible: https://claude.ai/share/a36beb8c-2530-41d7-88a3-f2fae1a92ae8
[10] Google Gemini, “Basic Search — Senku-IA”, 2025. [En línea]. Disponible: https://gemini.google.com/share/d47a275f6aae
[11] OpenCode, “Basic Search — Senku-IA”, 2025. [En línea]. Disponible: https://opncd.ai/share/68vADM52
[12] OpenCode, “Refactorización y memoria — Senku-IA”, 2026. [En línea]. Disponible: https://opncd.ai/share/xLLRjijG